Universidad de los Andes Colombia | Facultad de Economía

Taller 1
Taller de programación en R
Facultad de Economía, Universidad de los Andes

Entrega del script (Bloque Neón): lunes 31 de agosto de 2026, 11:59 p.m.
Sesión de evaluación: miércoles 2 de septiembre de 2026, en la clase magistral (Semana 5)

Motivación: entrenar como para una entrevista técnica

En el mundo de la programación existen plataformas diseñadas para practicar de cara a entrevistas técnicas de trabajo: *LeetCode*, *HackerRank*, *Codewars* o *Exercism*, entre otras. Estás plataformas funcionan usando un banco de ejercicios, en el cual se sube gradualmente de dificultad, utilizando código que funcione y que luego debe poder explicarse frente a un entrevistador.

Este taller busca simular exactamente esa dinámica. A continuación, encontrarán un banco de **20 ejercicios** organizados en **tres niveles de dificultad creciente** (Básico, Intermedio y Avanzado), construidos con el material de las semanas 1 a 3 del curso y un par de elementos de la semana 4. Su grupo deberá resolver los 20 ejercicios en un script de R y, en la sesión de evaluación, defender sus soluciones como en una entrevista técnica: **3 de los 20 ejercicios serán seleccionados al azar y ustedes deberán explicar, justificar y modificar su código en vivo.**

Aspectos generales

El presente taller busca que se apliquen los conceptos vistos durante las clases magistrales de las semanas 1 a 3 del curso (objetos, estructuras de datos, control de flujo y funciones), junto con un par de elementos de la semana 4 (data frames y la familia `apply`). Con los ejercicios propuestos se busca desarrollar una mejor comprensión del uso de R y fomentar el razonamiento lógico necesario para implementar correctamente los tópicos tratados.

- El taller debe ser desarrollado en los **grupos establecidos** para el desarrollo de los talleres (sin excepciones). **Sólo un integrante del grupo debe subir la entrega.**

- La entrega se realiza únicamente a través de **Bloque Neón**, a más tardar el **lunes 31 de agosto de 2026 a las 11:59 p.m.** No se aceptarán entregas por un medio distinto. Entregas por fuera de este horario conllevan las deducciones establecidas en el programa del curso.

- Conforme al programa del curso, la nota del Taller 1 corresponde **en su totalidad** a lo obtenido en la **sesión de evaluación** de la Semana 5 (miércoles 2 de septiembre). La nota es **grupal** y su rúbrica se encuentra al final de este documento. **La nota máxima del taller es 5,0.**

- La asistencia de **todos los integrantes** a la sesión de evaluación es obligatoria. Quien no asista obtendrá una nota individual de 0.0, salvo excusa válida según el Artículo 45 del reglamento[1].

Entregables

Cada grupo debe subir a Bloque Neón:

- **Un único script de R** (nombrado `Taller1_Grupo_#.R`) con la solución de los 20 ejercicios. El script debe correr de principio a fin **sin errores y sin intervención manual** (solo ejecutar), y debe estar organizado por niveles y ejercicios con comentarios claros (por ejemplo, `## Ejercicio 1`). Cuando un ejercicio pida responder o interpretar algo, la respuesta debe ir como comentario (`#`) inmediatamente debajo del código correspondiente. No se pide documento PDF.

- **Las conversaciones con herramientas de IA generativa**, en caso de haberlas usado, según lo descrito en la sección de uso de inteligencia artificial de este documento.

Tengan en cuenta que no solo se evaluará el resultado, sino el uso de las herramientas trabajadas en clase: **salvo que el enunciado del ejercicio lo autorice explícitamente, solo se permite el uso de funciones y librerías vistas en clase.**

Dinámica de la sesión de evaluación

La evaluación se realizará durante la sesión magistral del **miércoles 2 de septiembre de 2026** (Semana 5), en franjas horarias por grupo que el equipo pedagógico publicará previamente. La dinámica será la siguiente:

- **10 minutos antes de ingresar a su franja**, se le informará al grupo cuáles **3 de los 20 ejercicios** serán evaluados. Usen ese tiempo para repasarlos, no para reescribirlos. Se verificará que el script entregado corresponda al evaluado.

- Durante la sesión, el grupo deberá **correr su código en vivo**, explicar la lógica de cada solución y responder preguntas del equipo pedagógico (del tipo "¿qué hace esta línea?", "¿qué pasaría si…?", "modifiquen el código para…").

- Las modificaciones en vivo consistirán, por lo general, en **cambios pequeños a los parámetros de los ejercicios** (otra semilla, otro umbral, otra tasa, otro coeficiente del proceso generador de datos). Se espera que el grupo **anticipe el efecto antes de correr el código**, lo verifique corriéndolo y explique el resultado. Recuerden que los ejercicios exigen parametrizar los valores clave.
-
- **Todos los integrantes deben estar en capacidad de explicar cualquier parte del script.** El equipo pedagógico decidirá quién responde cada pregunta, por lo que repartirse los ejercicios sin entender el trabajo de los demás es una mala estrategia.

[1] En cuyo caso en que haya lugar, se aplicará el examen supletorio, de acuerdo en lo descrito en el programa del curso.

- Consideren este ejercicio como una oportunidad para enfrentarse a un escenario similar al de una entrevista técnica real. Como sugerencia, deberían entrenar la claridad, precisión y manejo del nerviosismo.

Sobre plagio y copia entre grupos

La copia o plagio en este trabajo está **totalmente prohibida**. Incurrir en esta práctica conlleva una nota de cero (0) en el taller, así como las sanciones correspondientes contempladas en el reglamento de estudiantes de la universidad.

En particular, sobre la **copia entre grupos**: la entrega de scripts idénticos o sospechosamente similares entre dos o más grupos conllevará una nota de cero (0) para **todos los grupos involucrados**, sin importar quién copió a quién, además del reporte correspondiente. Cada grupo es responsable de la custodia de su código por lo que no deben compartir sus scripts, ni parcial ni totalmente. Por supuesto discutir ideas o conceptos con otros grupos está bien.

Uso de inteligencia artificial (Nivel 3)

Bajo los lineamientos institucionales de la universidad, este taller se encuentra en **Nivel 3 de uso de inteligencia artificial generativa (IAG)**. Esto significa que está permitido apoyarse en herramientas de IAG (ChatGPT, Claude, Copilot, Gemini, etc.) durante el desarrollo del taller, bajo dos condiciones obligatorias:

- **Declarar su uso y enviar las conversaciones:** si utilizaron IAG, deben adjuntar en la entrega de Bloque Neón **la o las conversaciones completas** sostenidas con la herramienta (enlace público de la conversación o exportación en PDF). **Usar IAG sin adjuntar las conversaciones se considerará una violación a la política de uso de IA del curso.**

- **Comprender todo lo que entregan:** en la sesión de evaluación deberán explicar y modificar su código en vivo. No poder explicar código entregado se considerará evidencia de uso indiscriminado de IAG y se penalizará según la rúbrica.

Recomendación pedagógica: usen la IAG para entender errores y aclarar conceptos, no para generar soluciones completas. En la sesión de evaluación, igual que en una entrevista técnica, estarán solos frente a su código.

Banco de ejercicios

Instrucciones generales del banco:

- Resuelvan los 20 ejercicios en orden, en un único script, con un encabezado de comentario por nivel y por ejercicio.
- Los objetos creados en un ejercicio se reutilizan en ejercicios posteriores: usen exactamente los **nombres de objeto indicados** en cada enunciado.
- Está prohibido el uso de funciones automáticas que resuelvan directamente lo que el ejercicio pide construir manualmente (por ejemplo, `lm()`, `scale()` o similares cuando el ejercicio pide programación explícita).
- **Los ejercicios están parametrizados:** cada valor clave del enunciado (semilla, tamaño de muestra, umbrales, tasas, coeficientes) debe definirse **una única vez** como un objeto con nombre y el resto del código debe construirse a partir de esos objetos.

No escriban “números mágicos” repetidos por el script: en la sesión de evaluación el equipo pedagógico podrá cambiar el valor de uno o varios parámetros y todo el ejercicio debe seguir corriendo y produciendo resultados coherentes.

Nivel 1 — Básico (Ejercicios 1 a 8)

El calentamiento (*warm-up*): un ejercicio por concepto fundamental. En una entrevista, estos son los que hay que resolver rápido y sin dudar.

Ejercicio 1

Definan al inicio del script un objeto `semilla` cuyo valor será la suma de los códigos de los integrantes, divido por la cantidad de los mismos, y fijen la semilla con `set.seed(semilla)`. En un comentario, expliquen: (i) por qué es crucial fijar una semilla cuando el análisis involucra números aleatorios; (ii) qué pasaría con sus resultados si no la fijaran; y (iii) qué resultados de su script cambian y cuáles no si se reemplaza el valor de `semilla` por otro.

Ejercicio 2

Definan primero los parámetros del ejercicio:

```
n        <- 1000   # tamaño de la muestra
edad_min <- 18     # edad mínima
edad_max <- 65     # edad máxima
```

Y construyan a partir de ellos los siguientes objetos (todo el código debe depender de `n`, `edad_min` y `edad_max`, de modo que siga funcionando si estos valores cambian):

- **a.** `id`: vector numérico secuencial que tome valores desde 1 hasta `n`.
- **b.** `edades`: vector de edades entre `edad_min` y `edad_max` años, generado a partir de una distribución uniforme y redondeado a números enteros.
- **c.** `nombres`: vector de tipo carácter generado muestreando con reemplazo del siguiente banco de nombres:

```
banco_nombres <- c("Ana", "Bernardo", "Carlos", "Diana",
                   "Elena", "Fernando", "Gabriela",
                   "Hernán", "Inés", "Julián", "Karen",
                   "Luis", "Marta", "Nicolás", "Olga",
                   "Pedro", "Rosa", "Sara", "Tomás",
                   "Ulises", "Valentina", "William")
```

- **d.** `educacion`: factor **ordenado** con niveles “Primaria” < “Secundaria” < “Pregrado” < “Posgrado”, asignados aleatoriamente.

Verifiquen con `typeof()` y `class()` que cada objeto quedó del tipo esperado.

Ejercicio 3

Se les entrega el siguiente vector de ventas registrado como texto:

```
ventas_texto <- c("1200", "850", "930", "no reportado", "1100",
                  "745", "sin dato", "990", "1010", "875")
```

- **a.** Conviértanlo a numérico y expliquen en un comentario por qué R genera una advertencia y qué hace con los valores no convertibles.
- **b.** ¿Cuántos NA quedaron en el vector convertido? ¿Qué proporción del vector representan?
- **c.** Calculen el promedio de las ventas ignorando los valores faltantes.

Ejercicio 4
Trabajen sobre una copia de `edades` llamada `edades_copia`, usando los siguientes parámetros (todas las respuestas deben calcularse a partir de ellos, nunca con los números escritos a mano):

```
umbral_mayor <- 40   # umbral de edad del literal a.
lim_inf      <- 25   # límite inferior del literal b.
lim_sup      <- 35   # límite superior del literal b.
corte_na     <- 20   # corte de censura del literal c.
```

a. ¿Cuántos individuos tienen más de `umbral_mayor` años? ¿Qué proporción del total representan?
b. ¿Qué proporción de individuos tiene entre `lim_inf` y `lim_sup` años (ambos inclusive)?
c. Reemplacen por `NA` las edades menores a `corte_na` años en `edades_copia` y cuenten cuántos valores faltantes quedaron.
d. ¿En qué posiciones del vector están los individuos de exactamente `edad_max` años? (pista: `which()`).

Ejercicio 5
a. Muestren los niveles de `educacion` y expliquen en un comentario qué cambia al crear el factor con `ordered = TRUE` frente a crearlo sin este argumento.
b. ¿Qué proporción de individuos tiene un nivel educativo estrictamente mayor a "Secundaria"?
c. ¿Cuántos individuos hay en cada nivel educativo? (pueden usar `table()`).

Ejercicio 6
a. Con el vector `1:12`, construyan una matriz `M` de 4 filas por 3 columnas, llenada **por filas**.
b. Asignen nombres a la matriz: filas "obs_1" a "obs_4" y columnas "v1", "v2", "v3" (pueden apoyarse en `paste0()`).
c. Extraigan: la segunda fila completa, la tercera columna completa y la submatriz de las filas 1 a 2 con las columnas 2 a 3.
d. Calculen la transpuesta de `M` y el producto matricial `M %*% t(M)`. En un comentario: ¿qué dimensiones tiene el resultado y por qué?

Ejercicio 7
a. Construyan una lista `grupo_taller` que contenga: un vector de caracteres con los nombres de los integrantes del grupo, un vector numérico con el semestre que cursa cada integrante, un valor lógico `entrego_a_tiempo` y una lista anidada `contacto` con el correo de contacto y la ciudad.
b. Accedan al vector de nombres usando `$` y al segundo elemento de la lista usando `[[ ]]`.
c. Expliquen en un comentario la diferencia entre `grupo_taller[1]` y `grupo_taller[[1]]`.
d. Agreguen un elemento nuevo llamado `curso` con el valor "Taller de R" y eliminen el elemento `entrego_a_tiempo` usando `NULL`.

Ejercicio 8
Construyan un vector `frases` en el que cada elemento describa, en una sola oración con orden semántico adecuado, el nombre, la edad y el nivel educativo de cada individuo del Ejercicio 2. Ejemplo del formato esperado:

```
"Ana tiene 34 años y su nivel educativo es Pregrado."
```

Para esto, concatenen adecuadamente los vectores `nombres`, `edades` y `educacion` (concatenación vectorizada, sin loops). Muestren las primeras 5 frases. El ejemplo no debe ser usado para el ejercicio.

Nivel 2 — Intermedio (Ejercicios 9 a 15)

Control de flujo y funciones: código que toma decisiones y que se reutiliza. Las funciones que programen en este nivel serán los insumos del Nivel 3.

Ejercicio 9

Definan los cortes como parámetros y creen un vector `grupo_etario` que clasifique a cada individuo según `edades`: "joven" si es menor que `corte_joven`, "adulto" entre `corte_joven` y `corte_mayor` (inclusive) y "mayor" si supera `corte_mayor`. Usen `ifelse()` anidado. Reporten cuántos individuos hay en cada grupo. Intente utilizar una definición formal para este ejercicio[2].

```
corte_joven <- 30
corte_mayor <- 50
```

Ejercicio 10

Definan un objeto `valor_prueba` y escriban un bloque `if / else if / else` que imprima con `cat()`:

a. "El valor no es numérico" si `valor_prueba` no es numérico;
b. "El valor es numérico y par" si es numérico y divisible por 2;
c. "El valor es numérico e impar" en el caso restante.

Prueben el bloque con tres valores distintos: un número par, un número impar y un texto (muestren las tres corridas en el script). Pista: el operador módulo `%%` retorna el residuo de una división.

Ejercicio 11

Usando un loop `for` sobre los 1.000 individuos, impriman en pantalla únicamente las frases del Ejercicio 8, cuyos individuos que cumplan **simultáneamente** las tres condiciones:

- Su nombre NO empieza por vocal (A, E, I, O, U),
- su edad es par, y
- su edad es mayor al parámetro `edad_corte <- 55`.

Pista: `substr(texto, 1, 1)` extrae la primera letra de un texto. Escriban las condiciones de forma que el loop siga funcionando si en la sesión se cambia `edad_corte` o se invierte alguna condición (por ejemplo, que el nombre SÍ empiece por vocal).

Ejercicio 12

Definan los parámetros:

```
pob_inicial <- 50000   # población hoy
tasa        <- 0.023   # crecimiento anual
meta        <- 80000   # población objetivo
```

[2] Es decir, pueden citar algún documento o algún otra fuente que sirva para este fin.

Un municipio tiene hoy `pob_inicial` habitantes y su población crece a una tasa `tasa` cada año. Usando un bucle `while`, determinen cuántos años deben pasar para que la población supere la `meta`. Impriman el número de años y la población final alcanzada. En un comentario: ¿qué pasaría con su bucle si `tasa` fuera 0 y por qué hay que tener especial cuidado al programar con `while`? ¿Cómo protegerían su código ante ese caso?

Ejercicio 13

Programen una función `estadisticos_manuales()` que reciba un vector numérico y retorne una lista con el promedio y la desviación estándar, calculados **manualmente** (sin usar `mean()` ni `sd()`). La función debe:

- **a.** Detener la ejecución con `stop()` si el input no es numérico;
- **b.** lanzar una advertencia con `warning()` si el vector contiene NA, e ignorarlos en el cálculo;
- **c.** Incluir documentación breve (qué recibe, qué retorna, un ejemplo de uso).

Apliquen la función a `edades`[3]: ¿cuál es la edad promedio de su muestra y cuál su desviación estándar?

Ejercicio 14

Programen una función `estandarizar()` que transforme un vector numérico a media cero y desviación estándar uno. Dentro de esta función deben reutilizar `estadisticos_manuales()` del ejercicio anterior. Apliquen la función a `edades` para crear `edad_std` y verifiquen que el promedio del resultado es aproximadamente 0 y su desviación estándar aproximadamente 1. En un comentario: ¿por qué "aproximadamente" y no exactamente?

Ejercicio 15

Programen una función `lista_a_matriz()` que reciba una lista de vectores numéricos y retorne una matriz `X` cuya primera columna sea un vector de unos (la constante) y cuyas demás columnas sean los elementos de la lista, en orden. La función debe:

- **a.** Verificar con `stop()` que todos los elementos de la lista son numéricos y tienen la misma longitud;
- **b.** nombrar las columnas: "constante" para la primera y los nombres de la lista para las demás.

Pista: creen primero una matriz de unos con `matrix()` y llénenla columna por columna usando un loop e indexación `[ , j]`.

Nivel 3 — Avanzado (Ejercicios 16 a 20)

La ronda final de la entrevista: integrar simulación, álgebra matricial, funciones propias y manejo de datos. Los ejercicios 16 a 18 se construyen unos sobre otros.

Ejercicio 16

- **a.** Simulen, para los mismos 1.000 individuos: `salario`, distribuido uniformemente entre 1.400.000 y 12.000.000; y `experiencia`, con una distribución que ustedes elijan. Documenten en un comentario cuál distribución usaron y por qué es consistente con

[3] Recuerden seguir las reglas de documentación de una función, que pueden verla aquí.

una variable de experiencia laboral usual (por ejemplo, no debería tomar valores negativos).

b. Estandaricen con su función `estandarizar()` las variables `edades`, `salario` y `experiencia`, creando `edad_std`, `salario_std` y `exper_std`.

c. Definan los coeficientes del proceso generador de datos como parámetros:

```
beta_0 <- 2.0   # constante
beta_1 <- 1.2   # efecto de la edad estandarizada
beta_2 <- 0.8   # efecto del salario estandarizado
beta_3 <- 0.4   # efecto de la experiencia estandarizada
```

d. Generen un término de error `epsilon` con distribución normal estándar y construyan el vector `y` de `n` observaciones que siga el proceso generador de datos:

```
y = beta_0 + beta_1*edad_std + beta_2*salario_std +
    beta_3*exper_std + epsilon
```

El vector `y` debe construirse a partir de los objetos `beta_0` a `beta_3`: en la sesión podría pedírseles cambiar uno de los coeficientes y anticipar, antes de correr el código, qué debería pasar con las estimaciones del Ejercicio 18.

Ejercicio 17

Programen una función `mco_manual(X, y)` que estime por Mínimos Cuadrados Ordinarios los coeficientes `beta = (X'X)^(-1) X'y` usando álgebra matricial (`t()`, `%*%`, `solve()`). Está prohibido usar `lm()` o cualquier función automática de regresión. La función debe:

a. Funcionar para matrices `X` con cualquier número de columnas;

b. verificar con `stop()` que `X` es una matriz y que el número de filas de `X` coincide con la longitud de `y`;

c. retornar los coeficientes como un vector nombrado según los nombres de las columnas de la matriz de diseño.

Ejercicio 18

a. Usando `lista_a_matriz()`, construyan tres matrices de diseño: **Modelo A**: constante + edad estandarizada; **Modelo B**: Modelo A + salario estandarizado; **Modelo C**: Modelo B + experiencia estandarizada. Reporten las dimensiones de cada matriz.

b. Empleando un **loop**, estimen los tres modelos con `mco_manual()` y almacenen en una lista, para cada modelo: los coeficientes, el vector de residuos `e = y - X %*% beta` y la suma de cuadrados de los residuos `SCR = t(e) %*% e`.

c. En comentarios: ¿cómo cambia el coeficiente de la edad estandarizada entre los tres modelos y por qué? ¿Cuál modelo tiene menor SCR y por qué? ¿Cuál modelo captura mejor el proceso generador de datos que ustedes mismos simularon y qué evidencia tienen?

Ejercicio 19

a. Construyan `datos_taller`, un data frame con las columnas `id`, `nombres`, `edades`, `educacion` y `salario`.

b. Inspecciónenlo con `str()`, `head()` y `dim()`. En un comentario: ¿qué tipo tiene cada columna?

c. Definan un parámetro `n_faltantes <- 50`, seleccionen al azar `n_faltantes` posiciones con `sample()` y reemplacen por `NA` el salario de esas filas. Reporten el número y el porcentaje de valores faltantes por columna (ambos deben seguir siendo correctos si `n_faltantes` cambia).

d. Creen `datos_completos` únicamente con las filas completas (`complete.cases()`) y reporten cuántas filas se perdieron.
e. Ordenen `datos_completos` de mayor a menor salario y muestren las 5 primeras filas del resultado.

Ejercicio 20

a. Carguen la base `airquality` (incluida en R, con `data(airquality)`). ¿Cuántos valores faltantes hay en cada columna?
b. Usando `apply()`, calculen el promedio de cada columna ignorando los valores faltantes.
c. Programen una función `rango_manual()` que reciba un vector y retorne la diferencia entre su máximo y su mínimo (ignorando NA), y aplíquenla a todas las columnas con `apply()`.
d. En un comentario: ¿qué variable presenta el mayor rango y por qué el rango puede ser una medida engañosa de la variabilidad al comparar variables medidas en unidades distintas?

Rúbrica de evaluación (nota grupal)

La nota del taller se compone de los **3 ejercicios seleccionados al azar** (1,5 puntos cada uno) y de un componente **transversal** de la sesión (0,5 puntos), para un total de **5,0**. Los 3 ejercicios se informarán al grupo 10 minutos antes de ingresar a su franja.

Componente	**Puntaje**
Ejercicio evaluado 1	1,5
Ejercicio evaluado 2	1,5
Ejercicio evaluado 3	1,5
Desempeño transversal del grupo	0,5
Total	**5,0**

Criterios por ejercicio evaluado (se aplican de forma idéntica a cada uno de los 3 ejercicios):

Criterio	**¿Qué se evalúa?**	**Puntaje**
Corrección y ejecución del código	El código del ejercicio corre sin errores dentro del script entregado, produce el resultado correcto y utiliza las herramientas vistas en clase. Puntaje parcial si corre con ajustes menores o si el resultado es parcialmente correcto; 0,0 si no corre o resuelve un problema distinto al planteado.	0,6
Explicación de la lógica	El o los integrantes seleccionados explican qué hace cada parte del código y por qué eligieron esa solución, recorriéndolo línea a línea si se les pide. Se penaliza la explicación memorizada o superficial. No poder explicar código entregado se califica con 0,0 en este criterio y activa la revisión de la política de IAG.	0,5
Preguntas y modificaciones en vivo	Respuestas a preguntas del tipo “¿qué pasaría si…?” y manejo de cambios en los parámetros del ejercicio (semilla, umbrales, tasas, coeficientes): el grupo anticipa	0,4

Criterio	¿Qué se evalúa?	Puntaje
	el efecto del cambio, corre el código modificado y explica coherentemente el resultado. Si el código no soporta el cambio de un parámetro por tener valores escritos a mano, este criterio se reduce a la mitad.	
Subtotal por ejercicio		**1,5**

Criterios transversales (una sola vez por sesión):

Criterio	¿Qué se evalúa?	Puntaje
Comunicación	Claridad y fluidez al comunicar; uso preciso del vocabulario técnico (objeto, función, argumento, loop, matriz de diseño, etc.).	0,25
Dominio parejo del grupo	Todos los integrantes demuestran conocimiento del script completo. El equipo pedagógico escoge quién responde cada pregunta; un desbalance evidente entre integrantes reduce este puntaje.	0,25
Subtotal transversal		**0,5**

Notas sobre la calificación:

- La nota es grupal y aplica únicamente a los integrantes que asistan a la sesión. La inasistencia sin excusa válida implica una nota individual de 0,0 (Artículo 45 del reglamento; ver programa del curso para el examen supletorio).
- Se califica exclusivamente sobre el script entregado el lunes 31 de agosto en Bloque Neón.
- Los hallazgos de copia entre grupos o de uso de IAG no declarado se rigen por lo descrito en las secciones anteriores y priman sobre esta rúbrica.

Recomendaciones finales

- Pueden repartirse la escritura inicial de los ejercicios, pero asegúrense de que todos los integrantes entienden los 20: cualquiera puede ser preguntado por cualquiera de ellos.
- Practiquen en voz alta: tomen 2 o 3 ejercicios y explíquenselos entre ustedes como simulacro de entrevista ¿Qué pasaría si se modificaran algunas líneas de código?. La comunicación efectiva es parte de la nota.
- Antes de entregar, corran el script completo en una sesión limpia de R (ambiente vacío, `rm(list = ls())` o reiniciando R) para verificar que corre de principio a fin sin errores.
- El día de la sesión, tengan el script abierto y listo para ejecutar, y conéctense al menos 5 minutos antes de su franja horaria.
- Usen los 10 minutos previos para repasar los 3 ejercicios anunciados y decidir cómo organizarse, no para reescribir código.